NTA Data Analysis

Functional and Non-Functional Specification

CSC1049 – Year 3 Project

Kevin Biju – 22332903

Daragh Montague – 22375933

1. Introduction

1.1 Purpose

This document describes what our NTA Data Analysis project (Jump) is supposed to do and the quality standards it needs to meet. The basic idea is to give Irish commuters a way to check how reliable different bus, rail, and Luas routes actually are, rather than just showing them the fastest option on paper.

1.2 Scope

We pull in GTFS static timetables and real-time feeds from the NTA, store everything in PostgreSQL, run delay analysis and ML predictions on top of it, and serve it all through a React dashboard with a journey planner. The end goal is that someone can search for a route from A to B and see not just travel time but how likely it is to actually run on time.

1.3 Division of Work

Kevin built the frontend dashboard, the backend API, the route search logic, and the reliability visualisations. Daragh handled the data pipelines, GTFS ingestion, the analytics builder, and the ML model training. We both worked on the database schema and integration testing.

2. Functional Requirements

2.1 Data Collection and Integration

We continuously poll the NTA GTFS-Realtime API to get live TripUpdate data. Each response is stored as a raw snapshot with the protobuf payload, HTTP metadata, and a SHA256 hash so we can always go back and reproduce our analysis from the original data.

For static timetables, we periodically download the NTA GTFS ZIP file, validate that it has all the required files and the dates are formatted correctly, then load it into a staging schema before

swapping it into production. That way scheduled arrival times are always up to date for comparing against real-time predictions.

During the delay event construction phase, we join real-time stop predictions to their matching scheduled arrival times using trip_id, stop_id, and stop_sequence. This gives us a structured record of how late (or early) each stop arrival actually was.

2.2 Data Processing and Storage

Incoming data goes through preprocessing to deal with missing values, timestamp normalisation, and format differences between data sources. We use PostgreSQL with four separate schemas: gtfs_static for timetable data, gtfs_rt for raw snapshots, analytics for our derived delay events and model metrics, and gtfs_static_staging for safe atomic loads.

All schema changes go through Alembic migrations so we can track what changed and roll back if needed. We added indexes on the columns we query most often to keep things fast.

2.3 Delay Analysis and Prediction

From the delay events we compute reliability metrics per route and per stop: on-time percentage, average delay, median delay, and how many observations we have. On-time means arriving within 5 minutes of the scheduled time.

On the ML side, we train an XGBoost regression model to predict delays. The features are route, stop sequence, how far ahead the prediction was made, hour of day, and day of week. We retrain every few hours using the last 7 days of data, and we log MAE, median error, and what percentage of predictions land within 60 and 120 seconds each time.

We also looked into pulling in weather data as an extra feature since bad weather obviously affects delays, but this hasn't been fully integrated yet.

2.4 Visualisation Dashboard

The dashboard is a React + TypeScript SPA built with Vite. The home page has a hero section linking to the journey planner, a favourites section, an interactive Google Map showing stops around Dublin, and a pipeline status card that shows whether the data collection is running and how many snapshots and delay events we've processed.

Route pages show a big reliability gauge, filters for day of week and hour, a list of trips, and the route shape on the map. Stop pages show reliability broken down by each route that serves that stop. Everything works in dark mode and it's responsive on mobile.

Users can filter by route, day, hour, and transport type to drill down into the data they care about.

2.5 Journey Search and Route Planning

This is the main feature. Users type in where they're coming from and going to using an autocomplete that searches stop names. We find direct routes by checking which stops are near both points and whether any routes serve stops at both ends.

For transfers, the algorithm finds hub stops where two routes overlap, filters by geographic bounds, and builds two-leg itineraries through common transfer points. Results are sorted by reliability first, not just travel time, so users can see which journey is most likely to actually work out.

Each result shows estimated duration, per-leg reliability gauges, walk distances, and transfer info. Clicking a result opens a trip summary with a full breakdown of each leg.

We use the Google Maps API for all the mapping. Routes show up on the map with estimated times and reliability data alongside them.

2.6 System Management and Testing

All data retrieval and processing errors get logged for debugging. The pipeline orchestrator runs polling, analytics construction, and model retraining as one coordinated process.

We've got unit tests for the validation and transformation logic, and integration tests for the RT polling and static pipeline. Everything's version controlled on GitLab.

3. Non-Functional Requirements

3.1 Performance

The pipeline needs to process each incoming snapshot within 10 seconds so it doesn't fall behind. On the backend, we use indexed queries and Haversine distance calculations in SQL to keep API responses quick. The dashboard aims to render within 3 seconds for normal queries, and we use parallel fetching and loading states to make it feel faster than it is.

3.2 Scalability

The schema separation means we could add data from other cities or transport networks without major restructuring. Right now we're handling over 2.5 million delay events across thousands of route/stop combinations, and the database handles it fine with proper indexing.

3.3 Usability

We went with a glassmorphic design (frosted glass cards, backdrop blur, smooth transitions) to make it look modern and clean. The colour coding on reliability gauges is consistent throughout: green for good, amber for okay, red for bad. The search autocomplete uses debounced input so it doesn't lag while you're typing.

3.4 Reliability

If the NTA API goes down temporarily, the poller handles it through retries and conditional requests with ETags. We keep all raw snapshots so we can always reprocess if something goes wrong. The analytics builder uses timestamps rather than sequential IDs for tracking, which means data imported from different machines doesn't cause conflicts.

3.5 Maintainability

The frontend is split into pages, components, context, and an API client layer. The backend is organised into separate routers for routes, stops, trips, search, and analytics. Alembic handles all DB migrations. We use GitLab for version control and issue tracking.

3.6 Security

API keys for the NTA and Google Maps live in environment variables and aren't checked into git. Same for database credentials. CORS is set up to only allow requests from our frontend origin. We don't collect or store any personal data.

3.7 Accuracy

We validate predictions against test data using standard regression metrics. Each training run records MAE, median error, and what percentage of predictions fall within 60s and 120s thresholds. We track these over time so we can see if the model is getting better or worse.

3.8 Compliance and Ethics

We comply with the NTA API terms of use and Google Maps licensing. All the data we use is publicly available transport data, and we don't handle any personal information at any stage.